

Apéndice B

Manual de Instalación

Este apéndice proporciona un manual o guía de instalación de la aplicación mediante el uso de la herramienta *Docker Compose*. Permitirá desplegar la plataforma en un entorno local, totalmente configurable para las necesidades del usuario.

B.1. Requisitos previos a la instalación

Para comenzar con la instalación no se requiere de especial material, bastará con un ordenador con acceso a internet y que tenga *Docker*²⁶ y *Docker Compose*²⁷ previamente instalados. No es estrictamente necesario para el proyecto, pero es recomendable tener instalado también la herramienta *Git*²⁸.

B.2. Descarga del código fuente

El primer paso será descargar el código fuente de la aplicación, el cual se encuentra alojado en el repositorio <https://github.com/fcristallocagnoli/Quantum-Proxy>, en GitHub. Este puede obtenerse bien clonando el repositorio con el comando `git clone` o bien descargado el proyecto como archivo ZIP y descomprimiéndolo. Una vez hecho esto, entramos en la carpeta del proyecto `Quantum-Proxy/`.

B.3. Configuración del archivo `docker-compose.yml`

Antes de entrar en detalle de los pasos para levantar los servicios que conforman la aplicación, es necesario comprender la configuración existente en el fichero `docker-compose.yml`. De esta forma, el usuario podrá adaptar el funcionamiento de la aplicación a sus necesidades.

²⁶<https://docs.docker.com/engine/install/>

²⁷<https://docs.docker.com/compose/install/>

²⁸<https://git-scm.com/downloads>

A continuación (figura 3) se presenta el contenido de tal archivo de configuración. Iremos detallando servicio a servicio el significado de las variables de entorno y cómo personalizarlos.

```
services:
  database:
    image: mongo:6.0.13-jammy
    restart: always
    environment:
      DB_DATABASE: quantum-proxy-db
      DB_ADMIN_USER: ${DB_ADMIN_USER:-admin}
      DB_ADMIN_PASSWORD: ${DB_ADMIN_PASSWORD:-admin}
      DB_DUMMY_ACCOUNT: tfg.quantum.proxy@gmail.com
    ports:
      - "27017:27017"
    expose:
      - 27017
    volumes:
      - db-data:/data/db
      - ./backend/database/mongo-init.js:/docker-entrypoint-initdb.d/mongo-
        init.js:ro
  backend:
    build: ./backend
    environment:
      FRONTEND_URL: "http://frontend"

      DB_URI: "mongodb://admin:admin@database"
      DB_PORT: 27017

      DUMMY_ACCOUNT: tfg.quantum.proxy@gmail.com

      EMAIL_SENDER: tfg.quantum.proxy@gmail.com
      EMAIL_APP_PASSWORD: "tfbr yanj aiwj mvnv"
      EMAIL_ACCOUNT: ${EMAIL_ACCOUNT:-fcristallocalagnoli@uma.es}
    env_file:
      - path: ./aws-secrets.env
        required: false
    ports:
      - "8000:80"
```

```

    expose:
      - 8000
    depends_on:
      - database
frontend:
  build:
    context: ./frontend
    target: develop
    args:
      - BUILD_MODE=docker
  ports:
    - "80:4200"
  expose:
    - 80
  depends_on:
    - backend
volumes:
  db-data:

```

Source Code 3: Docker compose file

B.3.1. Servicio *database*

El servicio *database* define el comportamiento del contenedor que albergará la base de datos MongoDB en la que se basa la aplicación. Tal contenedor se creará a partir de la imagen oficial de MongoDB, *mongo*, concretamente la versión *6.0.13-jammy*.

El resto de opciones significan lo siguiente:

- **restart:** la opción ‘always’ indica al contenedor que se reinicie siempre que ocurra algún fallo interno.
- **ports:** mapea el puerto 27017 del host al puerto 27017 del contenedor.
- **expose:** sirve para documentar qué puertos están expuestos, el 27017 en este caso.
- **volumes:** define dos volúmenes de datos
 - El mapeo `db-data:/data/db` asocia un volumen persistente para almacenar los datos de la aplicación.

- La segunda definición mapea el fichero `mongo-init.js` (presente en la carpeta del backend) a la carpeta de `/docker-entry-point-initdb/`, donde se almacenará con acceso únicamente de lectura. Este fichero es de gran importancia, pues es el encargado de configurar la base de datos la primera vez que arranca el contenedor.

Pasamos ahora a las variables de entorno:

- **DB_DATABASE:** define el nombre de la base de datos por defecto.
- **DB_ADMIN_USER:** define el nombre de usuario administrador por defecto.
- **DB_ADMIN_PASSWORD:** define la contraseña de administrador por defecto.
- **DB_DUMMY_ACCOUNT:** crea un usuario con el email proporcionado como *dummy account*.

B.3.2. Servicio *backend*

El servicio *backend* define el contenedor que correrá la API REST que alimenta la plataforma web. Maneja diferentes *endpoints* para tratar con cuentas de usuario, proveedores y sistemas. Su imagen se construye a partir del Dockerfile ubicado en la carpeta `backend/` del proyecto (código en 1).

Detallando sus opciones tenemos que:

- **env_file:** especifica un fichero que contiene variables de entorno. En este caso, el archivo `aws-secrets.env`.
 - Las variables que se están inyectando son unas claves de AWS usadas para inicializar la información de este tercero. Si no se incluye el fichero `aws-secrets.env`²⁹, se omitirá de la ejecución.
- **ports:** mapea el puerto 8000 del host al puerto 80 del contenedor.
- **expose:** sirve para documentar qué puertos están expuestos, el 8000 en este caso.

²⁹Se proporciona un ejemplo en el repositorio GitHub: <https://github.com/fcristallocagnoli/Quantum-Proxy/blob/master/aws-secrets.env.sample>. Remover sufijo 'sample' y rellenar con valores.

- **depends_on**: permite controlar el orden en que se crearán los contenedores. Al indicar una dependencia, un servicio no podrá crearse hasta que aquellos de los que dependa se hayan iniciado correctamente. Este servicio depende de *database*.

Pasamos ahora a las variables de entorno:

- **FRONTEND_URL**: define la dirección donde se encontrará el servicio de *frontend*. El puerto de esta dirección deberá ser el que exponga el frontend, en este caso como es el 80 (predeterminado de HTTP), lo podemos omitir.
- **DB_URI**: define la URI que servirá de conexión a la base de datos. Los atributos entre ':' (admin:admin) deben corresponder a las claves **DB_ADMIN_USER** y **DB_ADMIN_PASSWORD** declaradas en el servicio de base de datos.
- **DB_PORT**: define el puerto de la conexión con la base de datos. Este debe corresponderse con el puerto expuesto en el servicio de la base de datos.
- **DUMMY_ACCOUNT**: define la cuenta de correo que se usará como *dummy account*³⁰.
- **EMAIL_SENDER** y **EMAIL_APP_PASSWORD**: son variables usadas para el servicio de emails automatizados³¹. **EMAIL_SENDER** define la cuenta en nombre de quien se enviarán los correos; **EMAIL_APP_PASSWORD** indica la contraseña de aplicación, necesaria para enviar emails de manera automática.
 - Para una mayor facilidad y simplicidad en el despliegue local, se proporcionan tanto la cuenta de correo como la contraseña de aplicación presentes en el fichero `docker-compose`. En un entorno real esto constituiría una pérdida de privacidad y confidencialidad, no se deben exponer ningún tipo de credenciales en el código (véase el uso de *secrets* para ocultar las variables de AWS)
- **EMAIL_ACCOUNT**: define la cuenta de correo a la que le serán enviados los correos con informes de posibles errores, dado el caso que falle algo en la aplicación.

³⁰Cuenta automatizada para obtener datos de proveedores y sistemas.

³¹Ya sea para enviar emails de verificación de cuenta, o para informar de errores en la aplicación

B.3.3. Servicio *frontend*

Por último, el servicio *frontend* define el contenedor que servirá la aplicación web desarrollada en Angular. La imagen asociada se construye a partir del Dockerfile presente en la carpeta *frontend/* del proyecto, el cual vemos a continuación en los fragmentos de código 4, 5 y 6. Donde a través de la configuración de *compose*, escogemos construir la etapa *develop* y con el argumento *BUILD_MODE* puesto a 'docker'.

Detallando sus opciones tenemos que:

- **ports:** mapea el puerto 80 del host al puerto 4200 del contenedor.
- **expose:** sirve para documentar qué puertos están expuestos, el 80 en este caso.
- **depends_on:** permite controlar el orden en que se crearán los contenedores. Al indicar una dependencia, un servicio no podrá crearse hasta que aquellos de los que dependa se hayan iniciado correctamente. Este servicio depende de *backend*.

Como el Dockerfile es bastante grande y algo complejo, lo fragmentaremos en sus diferentes etapas para poder comprenderlo de manera sencilla.

- **Build stage:** parte de una imagen de *Node* y se encarga de instalar las dependencias del proyecto Angular, así como de construir el proyecto en base al valor del argumento *BUILD_MODE*³². Esta etapa se ejecuta siempre.

```
# Selecciona Node como imagen base, versión lts-alpine
FROM node:lts-alpine AS build

# Establece el directorio de trabajo en /app
WORKDIR /app

# develop como build predeterminada
ARG BUILD_MODE=develop

# Copia los archivos package.json y package-lock.json
COPY package*.json .

# Instala las dependencias
RUN npm ci
```

³²Este argumento sirve para indicar internamente a la aplicación la ruta del backend.

```
# Copia el código del frontend
COPY . .
# Ejecuta el comando especificado en el argumento BUILD_MODE
RUN npm run ${BUILD_MODE}
```

Source Code 4: Etapa 'build', Dockerfile para el frontend

- **Develop stage:** parte de la etapa anterior *build*. Se encarga de instalar la CLI de Angular y servir la aplicación web. Esta etapa está pensada para el desarrollo.

```
# Selecciona la etapa build como base
FROM build as develop

# Instala Angular CLI
RUN npm install -g @angular/cli@17

# Expone el puerto 4200
EXPOSE 4200

# Indica el comando que se ejecutará al iniciar el contenedor
CMD [ "ng", "serve", "--host", "0.0.0.0" ]
```

Source Code 5: Etapa 'develop', Dockerfile para el frontend

- **Product stage:** parte de una imagen de *Nginx*, y se encarga copiar los archivos generados en la etapa *build* para servirlos como sitio web estático. Esta etapa está pensada para un despliegue optimizado usando *Nginx*.

```
# Selecciona Nginx como imagen base
FROM nginx:latest as product

# Copia el archivo de configuración de Nginx
COPY nginx.conf /etc/nginx/conf.d

# Copia los archivos generados en la etapa build a la carpeta de
  archivos estáticos de Nginx
COPY --from=build /app/dist/frontend/browser /usr/share/nginx/html

# Expone el puerto 80
```

EXPOSE 80

```
# Indica el comando que se ejecutará al iniciar el contenedor  
CMD [ "nginx", "-g", "daemon off;" ]
```

Source Code 6: Etapa 'product', Dockerfile para el frontend

B.4. Manejar los servicios con Docker Compose

Una vez se ha comprendido la configuración del fichero docker-compose y los correspondientes Dockerfiles, retomamos el proceso de instalación.

En este punto deberíamos encontrarnos dentro de la carpeta `Quantum-Proxy/`. Aquí, abriremos una terminal, y desplegaremos el entorno mediante comandos.

B.4.1. Construcción de los servicios

Trabajar con Docker Compose ofrece grandes ventajas a la hora del desarrollo, tal es así que únicamente precisaremos de un comando para arrancar y poner en marcha la aplicación.

```
$ docker compose up -d
```

Source Code 7: Comando para iniciar la aplicación

Una vez comience a ejecutarse el comando `7`, lo primero que hará será construir las imágenes correspondientes al backend y frontend, y descargar de DockerHub la imagen usada para la base de datos. Esto es debido a que la primera vez que se arranque la aplicación, Docker no tiene registro de ninguna de las imágenes de los contenedores, por lo que deberá generarlas antes de crear los contenedores. Puede demorar un poco, pero una vez se hayan obtenido las imágenes, no tendrá que volver a hacerlo.

Tan pronto como se creen las imágenes, Docker comenzará a levantar los contenedores uno a uno, siguiendo el orden de dependencias visto en `3` hasta que estén todos activos.

B.4.2. Acceso a la plataforma recién desplegada

Ya con todos los servicios corriendo, podremos acceder a los servicios mediante los siguientes enlaces:

- En <http://localhost> encontraremos la plataforma web.
- En <http://localhost:8000> encontraremos la API REST.
- En <http://localhost:27017> encontraremos la conexión a la base de datos MongoDB, a la cual podremos acceder con *mongosh* o *MongoDB Compass*, pues no dispone de interfaz web.

B.4.3. Terminación de los servicios

Si se desea acabar con las instancias creadas, basta con ejecutar el comando **8** a continuación. Dicho comando detendrá los contenedores “amablemente” y luego los eliminará.

```
$ docker compose down
```

Source Code 8: Comando para terminar la aplicación

Opcionalmente se puede añadir la opción **-v**, que también borrará el volumen creado para almacenar los datos. Si no se especifica, cada vez que se levante la aplicación los datos almacenados en el servicio *database* persistirán.

Y como ya se expuso anteriormente, las imágenes ya han sido generadas, lo único que se elimina son los contenedores. De este modo, si se vuelve a levantar la aplicación con el comando **7** se ejecutará mucho más rápido.

B.4.4. Siguientes pasos a tomar

Desplegar los servicios es solo el principio, una vez tenemos toda la aplicación corriendo en local, hay una serie de consideraciones a tener en cuenta para terminar su configuración:

1. La plataforma se inicializa con dos cuentas predeterminadas:

- Email: `admin@test.com`, Contraseña: `admintesting`
- Email: `<DB_DUMMY_ACCOUNT>`³³, Contraseña: `dummy-password`

La primera cuenta posee privilegios de administrador para cualquier tarea oportuna de administración. Mientras que la segunda supone la denominada *dummy account*. Esta

³³El contenido de dicha variable de entorno. Por defecto será `quantum.proxy@gmail.com`, pero se puede modificar por el usuario en el fichero `docker-compose`.

cuenta *dummy* deberá ser actualizada con las API keys correspondientes para poder obtener los datos de los proveedores y sus máquinas correctamente.

Platform	Key	Value		
ionq	TOKEN	<YOUR_VALUE>	-	+
ibm	TOKEN	<YOUR_VALUE>	-	+
amazon_braket	AWS_ACCESS_KEY_ID	<YOUR_VALUE>		
	AWS_SECRET_ACCESS_KEY	<YOUR_VALUE>		
	AWS_REGION	<YOUR_VALUE>	-	+

- +

Figura 39: Ejemplo de API keys requeridas para obtener los datos

2. Una vez rellenas las API keys, pueden inicializarse los sistemas desde el panel de administrador dentro de la plataforma.